ソフトウェア工学特論 講義資料

第6回 GitHubとリモートリポジトリ

- GitHubの概要とホスティングサービスの違い
- clone / push / pull によるリモート操作
- 認証方式（SSH / Personal Access Token）

目次

- GitHubの概要
- リモート操作の基本
- 認証と公開設定

GitHubの概要

今回の目的と到達目標

今回の目的

- リモートリポジトリの役割を理解する
- clone/push/pullを自分で実行できる
- 認証設定を済ませる

到達目標

- [R2-未到達] push/pull/cloneでリモート操作できる
- [R2-標準] GitHub運用の基礎を押さえる

ホスティングサービスの比較

サービス	特徴
GitHub	世界最大。OSSの中心地。本講座で使用
GitLab	自前ホスティングに強み。CI統合が豊富
Bitbucket	Atlassian製品と連携が強い

- いずれもgitプロトコルでやり取りする点は共通
- 差はWebUI・権限管理・CIなどの **周辺機能**
- 本講座のチーム開発は GitHub 上で行う

リモートリポジトリの役割

- チームメンバーが **同じ履歴を共有** する場所
- 自宅PC・学校PCなど複数環境の橋渡し
- バックアップとしての機能
- Issue / PR / コードレビュー等の運用の土台（次回）
- 万一ローカルを消してもリモートから復元できる

リモート操作の基本

基本コマンド

コマンド	役割
`git clone <url>`	リモートをローカルに複製
`git remote -v`	リモートURLの確認
`git push origin <branch>`	ローカル→リモートへ送信
`git pull`	リモート→ローカルへ取得＋マージ
`git fetch`	取得のみ（マージしない）

`origin` は既定のリモート名。

典型的なワークフロー

```
# 1. クローンして作業開始
git clone git@github.com:team/project.git
cd project

# 2. 変更してコミット
git add .
git commit -m "機能Aを追加"

# 3. リモートに反映
git push origin main

# 4. 他メンバーの変更を取り込む
git pull
```

push の前に pull で最新を取り込むのが安全。

認証と公開設定

認証方式

方式	用途	特徴
HTTPS + PAT	標準。どの環境でも	Personal Access Tokenを発行
SSH鍵	継続運用向け	鍵ペアを登録して自動認証

- パスワード認証は2021年に廃止
- PAT は **権限スコープを限定** して発行する
- SSH鍵は `~/.ssh/id_ed25519.pub` を GitHub に登録

演習で各自アカウント作成と認証設定を行う。

リポジトリの公開範囲

- **Public**：誰でも閲覧可能
  - チーム開発のテーマが公開前提なら選択
- **Private**：指定メンバーのみ
  - 学内プロジェクトや開発途中のもの
- **Collaborator**：Private リポジトリに個別招待するメンバー
- 第8週のチーム編成後、リポジトリとCollaboratorを設定する

今回のまとめ

- GitHubはgit履歴を共有するためのリモート基盤
- 基本操作：clone / push / pull / fetch
- 認証はPATまたはSSH鍵（パスワード認証は不可）
- 第8週からチーム用リポジトリを作成する

今回カバーしたMCC項目

- V-D-1 プログラミング
- V-D-4 コンピュータシステム

次回予告

- 第7回: GitHub Issue・Pull Request・ブランチ戦略